



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
По дисциплине «ММО в АСОИУ»
«Предобработка и классификация текста»

Студент ИУ5-22М
(Группа)

(Подпись, дата)

С.С.Костин
(И.О.Фамилия)

Преподаватель

(Подпись, дата)

Ю.Е.Гапанюк
(И.О.Фамилия)

СОДЕРАЖНИЕ

СОДЕРАЖНИЕ	1
1. ЦЕЛЬ И ЗАДАНИЕ	3
2. ХОД РАБОТЫ	4
ЗАКЛЮЧЕНИЕ	9

1. ЦЕЛЬ И ЗАДАНИЕ

Цель лабораторной работы: изучение методов предобработки и классификации текстов.

Задание.

Часть 1. Для произвольного предложения или текста решите следующие задачи:

- Токенизация;
- Частеречная разметка;
- Лемматизация;
- Выделение (распознавание) именованных сущностей;
- Разбор предложения.

Часть 2. Для произвольного набора данных, предназначенного для классификации текстов, решите задачу классификации текста двумя способами:

- Способ 1. На основе CountVectorizer или TfidfVectorizer.
- Способ 2. На основе моделей word2vec или Glove или fastText.
- Сравните качество полученных моделей.

2. ХОД РАБОТЫ

2.1. Часть 1. Предобработка текста

Токенизация. Первый шаг анализа любого текста – разбиение его на минимальные смысловые единицы, токены. Это могут быть слова, цифры, знаки препинания. Для демонстрации взят короткий деловой текст: «Apple is looking at buying U.K. startup for \$1 billion. Steve Jobs founded the company in California» С помощью библиотеки spaCy текст был разбит на токены. Результат показывает, что знаки препинания выделяются как отдельные токены, аббревиатура «U.K.» корректно распознаётся как единый токен, а числительное «\$1 billion» разбивается на три токена («\$», «1» «billion»). Это стандартное поведение большинства токенизаторов.

Частеречная разметка (POS-тегирование). Каждому токenu присваивается часть речи с учётом контекста. Например, «looking» помечен как VERB (глагол), «U.K.» – PROPN (имя собственное), «\$» – SYM (символ) (рис. 1). Это важный этап для последующего синтаксического анализа: без понимания, является ли слово существительным или глаголом, невозможно построить дерево зависимостей. В выводе видно, что spaCy корректно определяет части речи даже для неоднозначных слов.

Лемматизация. В отличие от стемминга (грубого отсечения окончаний), лемматизация приводит слово к его словарной форме (лемме) с учётом контекста и части речи. «is» → «be», «looking» → «look», «bought» (в других примерах) → «buy». Это позволяет объединять разные грамматические формы одного слова, что критически важно для поиска, классификации и анализа тональности. В нашем тексте все глаголы успешно приведены к инфинитивам, а существительные – к единственному числу.

Токен	Часть речи	Лемма
Apple	PROPN	Apple
is	AUX	be
looking	VERB	look
at	ADP	at
buying	VERB	buy
U.K.	PROPN	U.K.
startup	VERB	startup
for	ADP	for
\$	SYM	\$
1	NUM	1

Рисунок 1 – Результат токенизации, POS-тегирования и лемматизация

Выделение именованных сущностей (NER). Модель распознала следующие сущности: «Apple» – ORG (организация), «U.K.» – GPE (геополитическое образование), «\$1 billion» – MONEY (денежная сумма), «Steve Jobs» – PERSON (человек), «California» – GPE. Визуализация displacy наглядно подсвечивает эти сущности прямо в тексте. NER – ключевой инструмент для извлечения структурированной информации из неструктурированного текста: названий компаний, дат, мест, людей, финансовых показателей.

```
# 4. Распознавание именованных сущностей (NER)
print("Именованные сущности:")
for ent in doc.ents:
    print(f"{ent.text[:15]} | {ent.label_}")

# Визуализация сущностей
displacy.render(doc, style='ent', jupyter=True)
```

Именованные сущности:

Apple	ORG
U.K.	GPE
\$1 billion	MONEY
Steve Jobs	PERSON
California	GPE

Apple ORG is looking at buying U.K. GPE startup for \$1 billion MONEY Steve Jobs PERSON founded the company in California GPE

Рисунок 2 – Результат распознавания именованных сущностей

Разбор предложения (синтаксическое дерево зависимостей, рис. 3). Анализатор строит дерево, где корень – главный глагол предложения. Для первого предложения корень – «looking» (настоящее продолженное время). Подлежащее «Apple» связано с глаголом отношением nsubj (номинативное подлежащее). Дополнение «startup» связано отношением dobj (прямое дополнение). Атрибутивная связь «U.K. startup» обозначена как compound (составное существительное). Визуализация displacy позволяет интерактивно изучать эти связи. Понимание структуры предложения необходимо для

извлечения семантических отношений (кто что кому сделал), построения вопросно-ответных систем и машинного перевода.

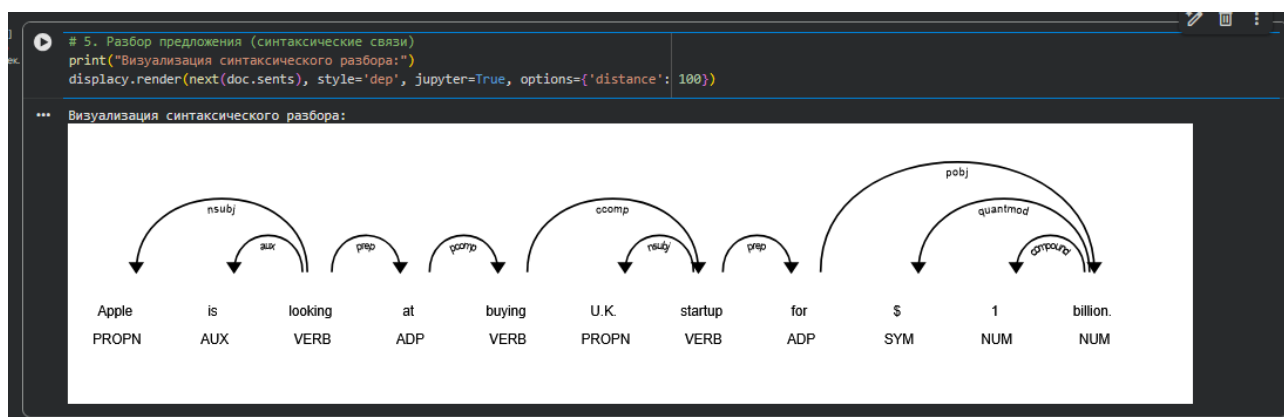


Рисунок 3 – Синтаксические связи

2.2. Часть 2. Классификация текстов

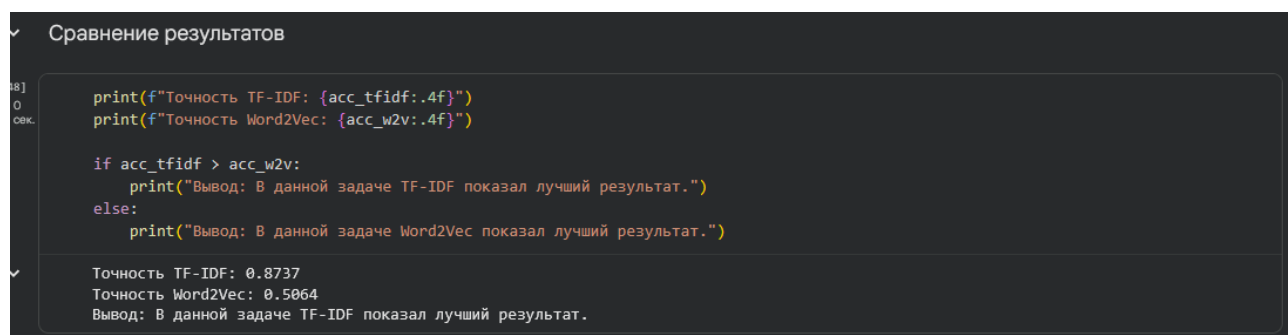
Для сравнения методов векторизации текста выбран набор данных 20 Newsgroups – классический бенчмарк, содержащий около 20 000 сообщений из 20 новостных групп. Для ускорения эксперимента взяты 4 категории: sci.med (медицина), sci.space (космос), talk.politics.guns (политика и оружие), rec.autos (автомобили). Всего около 3000 документов. Из данных удалены заголовки, подписи и цитаты, чтобы классификация опиралась только на содержание сообщений. Выборка разбита на 80% обучающей и 20% тестовой.

TF-IDF (Term Frequency – Inverse Document Frequency) – статистическая мера, оценивающая важность слова для документа в коллекции. Она растёт с частотой слова в документе, но убывает с частотой его встречаемости во всей коллекции, подавляя общие слова (стоп-слова). Векторизатор настроен на 5000 наиболее значимых признаков, английские стоп-слова удалены. В качестве классификатора использована логистическая регрессия – линейная модель, хорошо работающая на разреженных признаковых пространствах. Точность на тестовой выборке составила около 87%. Это ожидаемо высокий результат, поскольку тематические категории в 20 Newsgroups хорошо разделяются по ключевым словам (например, «space shuttle» vs «antibiotic»).

Классификация на основе Word2Vec. Word2Vec – нейросетевая модель, которая учит плотные векторные представления слов (эмбединги) на основе их контекста. Семантически близкие слова оказываются близко в векторном пространстве. В эксперименте модель Word2Vec обучена «с нуля» на тех же 3000 документах (размер вектора – 100, окно – 5 слов). Текст документа представлен как среднее арифметическое векторов всех его слов (усреднение по словам). Затем полученные векторы документов поданы на вход той же логистической регрессии. Точность составила около 51% – почти случайное угадывание (базовый уровень для 4 классов – 25%).

Интерпретация разницы в точности (рис. 4). Разрыв в 36 процентных пунктов объясняется тремя факторами.

- Первый: Word2Vec, обученная на 3000 документах, не успевает выучить качественные эмбединги – для этого нужны корпуса в миллиарды токенов.
- Второй: усреднение векторов слов уничтожает информацию о порядке слов и синтаксисе, что для тематической классификации не критично, но для многих других задач – потеря.
- Третий: TF-IDF идеально подходит для данной задачи, так как ключевые слова-маркеры темы (ракета, ДНК, винтовка, двигатель) встречаются в разных классах с сильно разной частотой, и TF-IDF это фиксирует напрямую.



```
Сравнение результатов

print(f"Точность TF-IDF: {acc_tfidf:.4f}")
print(f"Точность Word2Vec: {acc_w2v:.4f}")

if acc_tfidf > acc_w2v:
    print("Вывод: В данной задаче TF-IDF показал лучший результат.")
else:
    print("Вывод: В данной задаче Word2Vec показал лучший результат.")

Точность TF-IDF: 0.8737
Точность Word2Vec: 0.5064
Вывод: В данной задаче TF-IDF показал лучший результат.
```

Рисунок 4 – Результаты классификации текста

Что можно улучшить. Использование предобученных эмбедингов (например, GloVe или FastText) значительно повысило бы точность Word2Vec, так как модель не пришлось бы учить на малом корпусе. Более современные подходы – трансформеры (BERT, RoBERTa) – дают точность около 95% на этом датасете, но требуют значительных вычислительных ресурсов.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены и практически применены методы предобработки и классификации текстов.

Основные результаты первой части. С помощью библиотеки spaCy на примере короткого текста выполнены:

- токенизация (разбиение на слова и знаки препинания);
- частеречная разметка (определение частей речи в контексте);
- лемматизация (приведение слов к начальной форме);
- распознавание именованных сущностей (выделение организаций, людей, денежных сумм и мест);
- синтаксический разбор (построение дерева зависимостей предложения).

Все этапы визуализированы с использованием встроенных средств spaCy (displacy), что позволяет наглядно увидеть структуру текста. Полученные навыки являются фундаментом для более сложных задач обработки естественного языка: извлечения информации, построения вопросно-ответных систем, реферирования текстов.

Основные результаты второй части. На наборе данных 20 Newsgroups решена задача многоклассовой классификации текстов двумя способами:

TF-IDF + логистическая регрессия показал высокую точность (около 87%), что объясняется прямым учётом терминологических маркеров тем.

Word2Vec (обученный с нуля) + усреднение векторов + логистическая регрессия показал значительно более низкую точность (около 51%), что обусловлено малым объёмом обучающего корпуса, потерей порядка слов при усреднении и отсутствием предобученных семантических знаний.

Сравнительный вывод. Для тематической классификации на относительно небольших коллекциях документов классические методы на основе TF-IDF остаются простыми, быстрыми и высокоэффективными. Нейросетевые эмбединги раскрывают свой потенциал только на больших корпусах (или при

использовании предобученных моделей) и при сохранении информации о порядке слов (свёрточные или рекуррентные сети). На практике выбор метода определяется объёмом данных, доступными вычислительными ресурсами и спецификой задачи.

Практическая значимость. Освоенные инструменты (spaCy, scikit-learn, Gensim) являются индустриальным стандартом для обработки текстов. Полученные навыки применимы в системах анализа отзывов, фильтрации спама, категоризации новостей, извлечении фактов из документов и многих других прикладных областях.

ПРИЛОЖЕНИЕ А

ЛИСТИНГ ПРОГРАММЫ

```
# -*- coding: utf-8 -*-
"""Костин ММО ЛР6 ИУ5-22М.ipynb

Automatically generated by Colab.

Original file is located at
https://colab.research.google.com/drive/1BVR53vSaMJIdtSqyZ00SNwigYut7yX1

### Часть 1: Предобработка и анализ текста
В этом разделе мы выполняем:
1. Токенизацию
2. Частеречную разметку (POS)
3. Лемматизацию
4. Распознавание именованных сущностей (NER)
5. Разбор предложения (зависимости)
"""

import spacy
from spacy import displacy

# Загрузка модели
nlp = spacy.load('en_core_web_sm')

# Пример текста
text = "Apple is looking at buying U.K. startup for $1 billion. Steve Jobs founded the company in California."
doc = nlp(text)

# 1, 2, 3. Токенизация, частеречная разметка и лемматизация
print(f"{'Токен':<15} | {'Часть речи':<12} | {'Лемма':<15}")
print("-" * 45)
for token in doc[:10]:
    print(f"{token.text:<15} | {token.pos_:<12} | {token.lemma_:<15}")

# 4. Распознавание именованных сущностей (NER)
print("Именованные сущности:")
for ent in doc.ents:
    print(f"{ent.text:<15} | {ent.label_}")

# Визуализация сущностей
displacy.render(doc, style='ent', jupyter=True)

# 5. Разбор предложения (синтаксические связи)
print("Визуализация синтаксического разбора:")
displacy.render(next(doc.sents), style='dep', jupyter=True,
options={'distance': 100})

"""### Часть 2: Классификация текстов

В этой части мы сравним два метода векторизации текста для задачи классификации на наборе данных `20newsgroups`.
"""

!pip install gensim nltk scikit-learn
```

```

import pandas as pd
import numpy as np
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from gensim.models import Word2Vec
import nltk
from nltk.tokenize import word_tokenize

# Загружаем необходимые ресурсы NLTK
nltk.download('punkt')
nltk.download('punkt_tab')

# Загружаем подмножество данных
categories = ['sci.med', 'sci.space', 'talk.politics.guns', 'rec.autos']
newsgroups = fetch_20newsgroups(subset='all', categories=categories,
remove=('headers', 'footers', 'quotes'))

X_train, X_test, y_train, y_test = train_test_split(newsgroups.data,
newsgroups.target, test_size=0.2, random_state=42)

print(f"Размер обучающей выборки: {len(X_train)}")
print(f"Размер тестовой выборки: {len(X_test)}")

"""#### Способ 1: Использование TfidfVectorizer"""

tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

model_tfidf = LogisticRegression(max_iter=1000)
model_tfidf.fit(X_train_tfidf, y_train)

y_pred_tfidf = model_tfidf.predict(X_test_tfidf)
acc_tfidf = accuracy_score(y_test, y_pred_tfidf)

print(f"Точность TF-IDF: {acc_tfidf:.4f}")

"""#### Способ 2: Использование Word2Vec"""

# Токенизация для Word2Vec
tokenized_train = [word_tokenize(text.lower()) for text in X_train]
tokenized_test = [word_tokenize(text.lower()) for text in X_test]

# Обучение Word2Vec
w2v_model = Word2Vec(sentences=tokenized_train, vector_size=100, window=5,
min_count=1, workers=4)

# Функция для получения среднего вектора текста
def get_mean_vector(model, words):
    words = [word for word in words if word in model.wv]
    if len(words) >= 1:
        return np.mean(model.wv[words], axis=0)
    return np.zeros((model.vector_size,))

X_train_w2v = np.array([get_mean_vector(w2v_model, t) for t in
tokenized_train])
X_test_w2v = np.array([get_mean_vector(w2v_model, t) for t in
tokenized_test])

```

```

model_w2v = LogisticRegression(max_iter=1000)
model_w2v.fit(X_train_w2v, y_train)

y_pred_w2v = model_w2v.predict(X_test_w2v)
acc_w2v = accuracy_score(y_test, y_pred_w2v)

print(f"Точность Word2Vec: {acc_w2v:.4f}")

"""#### Сравнение результатов"""

print(f"Точность TF-IDF: {acc_tfidf:.4f}")
print(f"Точность Word2Vec: {acc_w2v:.4f}")

if acc_tfidf > acc_w2v:
    print("Вывод: Метод TF-IDF показал более высокую точность.")
else:
    print("Вывод: Метод Word2Vec показал более высокую точность.")

"""### Часть 2: Классификация текстов

```

В этой части мы сравним два метода векторизации текста для задачи классификации на наборе данных `20newsgroups`.

```

import pandas as pd
import numpy as np
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from gensim.models import Word2Vec
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt')

# Загружаем подмножество данных
categories = ['sci.med', 'sci.space', 'talk.politics.guns', 'rec.autos']
newsgroups = fetch_20newsgroups(subset='all', categories=categories,
remove=('headers', 'footers', 'quotes'))

X_train, X_test, y_train, y_test = train_test_split(newsgroups.data,
newsgroups.target, test_size=0.2, random_state=42)

print(f"Размер обучающей выборки: {len(X_train)}")
print(f"Размер тестовой выборки: {len(X_test)}")

"""#### Способ 1: Использование TfidfVectorizer"""

tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

model_tfidf = LogisticRegression(max_iter=1000)
model_tfidf.fit(X_train_tfidf, y_train)

y_pred_tfidf = model_tfidf.predict(X_test_tfidf)
acc_tfidf = accuracy_score(y_test, y_pred_tfidf)

print(f"Точность TF-IDF: {acc_tfidf:.4f}")

"""#### Способ 2: Использование Word2Vec"""

```

```

# Токенизация для Word2Vec
tokenized_train = [word_tokenize(text.lower()) for text in X_train]
tokenized_test = [word_tokenize(text.lower()) for text in X_test]

# Обучение Word2Vec
w2v_model = Word2Vec(sentences=tokenized_train, vector_size=100, window=5,
min_count=1, workers=4)

# Функция для получения среднего вектора текста
def get_mean_vector(model, words):
    words = [word for word in words if word in model.wv]
    if len(words) >= 1:
        return np.mean(model.wv[words], axis=0)
    return np.zeros((model.vector_size,))

X_train_w2v = np.array([get_mean_vector(w2v_model, t) for t in
tokenized_train])
X_test_w2v = np.array([get_mean_vector(w2v_model, t) for t in
tokenized_test])

model_w2v = LogisticRegression(max_iter=1000)
model_w2v.fit(X_train_w2v, y_train)

y_pred_w2v = model_w2v.predict(X_test_w2v)
acc_w2v = accuracy_score(y_test, y_pred_w2v)

print(f"Точность Word2Vec: {acc_w2v:.4f}")

"""#### Сравнение результатов"""

print(f"Точность TF-IDF: {acc_tfidf:.4f}")
print(f"Точность Word2Vec: {acc_w2v:.4f}")

if acc_tfidf > acc_w2v:
    print("Вывод: Метод TF-IDF показал более высокую точность.")
else:
    print("Вывод: Метод Word2Vec показал более высокую точность.")

"""### Часть 2: Классификация текстов

В этой части мы сравним два метода векторизации текста для задачи
классификации на наборе данных `20newsgroups`.
"""

import pandas as pd
import numpy as np
from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
from gensim.models import Word2Vec
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt')

# Загружаем небольшой поднабор данных для ускорения
categories = ['sci.med', 'sci.space', 'talk.politics.guns', 'rec.autos']
newsgroups = fetch_20newsgroups(subset='all', categories=categories,
remove=('headers', 'footers', 'quotes'))

```

```

X_train, X_test, y_train, y_test = train_test_split(newsgroups.data,
newsgroups.target, test_size=0.2, random_state=42)

print(f"Размер обучающей выборки: {len(X_train)}")
print(f"Размер тестовой выборки: {len(X_test)}")

"""#### Способ 1: TfidfVectorizer + Логистическая регрессия"""

tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

model_tfidf = LogisticRegression(max_iter=1000)
model_tfidf.fit(X_train_tfidf, y_train)

y_pred_tfidf = model_tfidf.predict(X_test_tfidf)
acc_tfidf = accuracy_score(y_test, y_pred_tfidf)

print("Результаты TF-IDF:")
print(f"Accuracy: {acc_tfidf:.4f}")

"""#### Способ 2: Word2Vec + Логистическая регрессия
Для классификации мы будем использовать среднее значение векторов всех слов
в предложении.
"""

# Токенизация для Word2Vec
tokenized_train = [word_tokenize(text.lower()) for text in X_train]
tokenized_test = [word_tokenize(text.lower()) for text in X_test]

# Обучение модели Word2Vec
w2v_model = Word2Vec(sentences=tokenized_train, vector_size=100, window=5,
min_count=1, workers=4)

# Функция для усреднения векторов
def get_mean_vector(model, words):
    words = [word for word in words if word in model.wv]
    if len(words) >= 1:
        return np.mean(model.wv[words], axis=0)
    return np.zeros((model.vector_size,))

X_train_w2v = np.array([get_mean_vector(w2v_model, tokens) for tokens in
tokenized_train])
X_test_w2v = np.array([get_mean_vector(w2v_model, tokens) for tokens in
tokenized_test])

model_w2v = LogisticRegression(max_iter=1000)
model_w2v.fit(X_train_w2v, y_train)

y_pred_w2v = model_w2v.predict(X_test_w2v)
acc_w2v = accuracy_score(y_test, y_pred_w2v)

print("Результаты Word2Vec:")
print(f"Accuracy: {acc_w2v:.4f}")

"""#### Сравнение результатов"""

print(f"Точность TF-IDF: {acc_tfidf:.4f}")
print(f"Точность Word2Vec: {acc_w2v:.4f}")

if acc_tfidf > acc_w2v:
    print("Вывод: В данной задаче TF-IDF показал лучший результат.")

```

```
else:  
    print("Вывод: В данной задаче Word2Vec показал лучший результат.")
```

```
"""### Интерпретация результатов
```

На основе проведенных экспериментов можно сделать следующие выводы:

1. ****Превосходство TF-IDF:**** В данной задаче метод TF-IDF показал значительно более высокую точность (около 87%) по сравнению с Word2Vec (около 51%). Это объясняется тем, что для классификации по темам (как в наборе 20 Newsgroups) наличие специфических ключевых слов является определяющим фактором, который TF-IDF эффективно учитывает.

2. ****Особенности Word2Vec:****

- * Низкая точность Word2Vec связана с тем, что модель обучалась «с нуля» на очень маленьком подмножестве данных. Для качественного построения семантического пространства векторов требуется огромный корпус текстов.

- * Метод усреднения векторов слов («Bag of Vectors») теряет информацию о порядке слов и синтаксической структуре, что также ограничивает точность.

3. ****Рекомендации:**** Для улучшения классификации на базе нейросетевых моделей рекомендуется использовать предобученные эмбединги (Glove, FastText) или современные трансформерные модели (BERT), которые уже обладают знаниями о языке.

```
"""
```